Ai Assisted Coding

Ht.No:2303A52493

Lab:19.4

Batch:50

Task 1 – JavaScript to Python (Array Processing Logic)

Scenario

A frontend application written in JavaScript processes user scores. The backend team wants the same logic implemented in Python for analytics processing.

Task: Translate a JavaScript function that:

• Takes an array of numbers. Filters values greater than 50. Returns the average of filtered values

Instructions: Prompt AI to convert JavaScript array methods (filter, reduce) into equivalent Python logic. Ensure list comprehension or Pythonic style is used. Compare outputs of both implementations with sample input. Document differences in handling arrays between JS and Python.

Prompt:

Convert the cod from python to javascript for a A frontend application written in JavaScript processes user scores. The backend team wants the same logic implemented in Python for analytics processing. Takes an array of numbers. Filters values greater than 50. Returns the average of filtered values. AI to convert JavaScript array methods (filter, reduce) into equivalent Python logic. Ensure list comprehension or Pythonic style is used. Compare outputs of both implementations with sample input. Document differences in handling arrays between JS and Python.

Code: in convert.py file

```
javascript_code = """

function calculateAverage(numbers) {

   const filteredNumbers = numbers.filter(num => num > 50);

   if (filteredNumbers.length === 0) {

      return null;  }

   const average =

      filteredNumbers.reduce((sum, num) => sum + num, 0) /

      filteredNumbers.length;

   return average; }

const numbers = [10, 60, 30, 80];

const average = calculateAverage(numbers);

if (average !== null) {

   console.log(`The average of numbers greater than 50 is: ${average}`);
```

```
} else {
    console.log("No numbers greater than 50 were found."); }
"""
with open("average.js", "w") as f:
    f.write(javascript_code)
print("✅ average.js file created!")
```

Code: in average.js file

```
function calculateAverage(numbers) {
    const filteredNumbers = numbers.filter(num => num > 50);
    if (filteredNumbers.length === 0) {
        return null;  }
    const average =
        filteredNumbers.reduce((sum, num) => sum + num, 0) /
        filteredNumbers.length;
    return average;  }
const numbers = [10, 60, 30, 80];
const average = calculateAverage(numbers);
if (average !== null) {
    console.log(`The average of numbers greater than 50 is: ${average}`);
} else {
    console.log("No numbers greater than 50 were found.");
}
```

Commands in cmd:

```
python convert.py
node average.js
```

Output:

```
C:\Users\mouni\Downloads\javascriptpractice>python convert.py
✅ average.js file created!

C:\Users\mouni\Downloads\javascriptpractice>node average.js
The average of numbers greater than 50 is: 70
```

Task 2 – Python OOP to C++ Class Translation

Scenario: A data modeling module is written in Python using OOP. The system is being migrated to a high-performance C++ environment.

Task: Translate a Python class that: Contains private attributes. Uses getter/setter methods. Includes a method that calculates a derived value

Instructions: Convert the Python class into a C++ class.

• Ensure:Proper access modifiers. Constructor handling. Correct data types. Compile and test the C++ version. Compare how encapsulation differs between Python and C++.

Prompt:

Python OOP to C++ Class Translation. : A data modeling module is written in Python using OOP. The system is being migrated to a high-performance C++ environment. Translate a Python class that: Contains private attributes. Uses getter/setter methods. Includes a method that calculates a derived value.

Code: in employee.py file

```
class Employee:
    def __init__(self, name, salary):
        self.__name = name
        self.__salary = salary
    def get_name(self):
        return self.__name
    def annual_salary(self):
        return self.__salary * 12
emp = Employee("Mounika", 30000)
print("Name:", emp.get_name())
print("Annual Salary:", emp.annual_salary())
```

Code: in employee.cpp file

```
#include <iostream>
using namespace std;
class Employee {
private:
    string name;
    double salary;
public:
```

```
    Employee(string n, double s) {
        name = n;
        salary = s;  }
    string getName() {
        return name; }
    double getSalary() {
        return salary;   }
    void setSalary(double s) {
        if (s > 0) {
            salary = s;
        }
    }
    double annualSalary() {
        return salary * 12;  } };
int main() {
    Employee emp("Mounika", 30000);
    cout << "Name: " << emp.getName() << endl;
    cout << "Annual Salary: " << emp.annualSalary() << endl;
    return 0; }
```

Commands in cmd:

python employee.py

Output:

```
C:\Users\mouni\Downloads\javascriptpractice>python employee.py
Name: Mounika
Annual Salary: 360000
```